IBM Innovation Centre for Education

NCS NihonCommunicationSolutions

YENEPOYA
(Deemed To Be University)

YENEPOYA UNIVERSITY

LOW LEVEL DESIGN (LLD) DOCUMENT
ON
ANOMALY DETECTION FOR HEALTHCARE TRANSACTIONS

Student Name: K Mohammad Irfan Hussain

Registration No: 23BSAMD012

Campus ID: 26998

Project Code: PRJN26-229

Industry Mentor: Sumit Kumar Shukla

Guided By: Ms. Fathimath Raheema

TABLE OF CONTENTS

1. INTRODUCTION 03
1.1 Document Purpose 03
1.2 Intended Audience 03

2. LLD SCOPE AND RUNTIME FLOW 04
2.1 Execution Pipeline 04

3. INPUT/OUTPUT DATA STRUCTURES & CONTRACTS 05
3.1 Incoming Transaction Payload (API Input) 05
3.2 Provider Context (Database Fetch) 05
3.3 API Response Payload (System Output) 06

4. FEATURE CONSTRUCTION LOGIC 06
4.1 Static Features (Spatial Context) 07
4.2 Temporal Features (Sequential Context) 08

5. MACHINE LEARNING MICRO-ARCHITECTURE 08
5.1 Static Model Logic (Isolation Forest) 08
5.2 Temporal Model Logic (LSTM Autoencoder) 09

6. EXPLAINABILITY TRIGGER (XAI) 09
6.1 Conditional Execution Logic 10
6.2 Output Formatting 10
6.3 Error Handling 10

7. HYBRID SCORE COMPUTATION & DECISION LOGIC 11
7.1 Normalization and Fusion 11
7.2 Threshold Application 11

8. DEPLOYMENT & DOCKER CONFIGURATION 12
8.1 Containerization Strategy 12
8.2 Microservice Networking 12

9. CONCLUSION 13

1. INTRODUCTION

1.1 Document Purpose

The Low-Level Design (LLD) document serves as the definitive technical blueprint for the internal mechanics of the Anomaly Detection for Healthcare Transactions system (PRJN26-229). While the High-Level Design (HLD) outlines the general system architecture and user interactions, this document dives into the granular, programmatic specifics.

It defines the exact JSON data contracts, the mathematical formulas driving the Machine Learning micro-architecture, the conditional logic for Explainable AI (SHAP) execution, and the containerized deployment strategy. This document is designed to guide the development, testing, and deployment phases by eliminating architectural ambiguity.

1.2 Intended Audience

This document is intended for:

- Project Reviewers & Evaluators: To demonstrate the rigorous technical logic, microservice orchestration, and mathematical foundations underlying the anomaly detection pipeline for comprehensive academic assessment.
- Developers & Software Engineers: To provide precise programmatic specifications for building the FastAPI backend, developing the Streamlit frontend, and integrating the SQLite database.
- Data Scientists & ML Engineers: To document the exact feature engineering logic, Isolation Forest parameters, LSTM sequence tensor structures, and SHAP conditional triggers.
- System Architects & Future Maintainers: To serve as a definitive reference guide for future system scalability, codebase maintenance, or transitioning the Docker containers into a live cloud environment.

2. LLD SCOPE AND RUNTIME FLOW

This section details the internal mechanics of the Anomaly Detection inference pipeline. The system operates strictly in real-time, evaluating each incoming transaction by contextualizing it against the specific healthcare provider's historical billing, demographic patterns, and clinical indicators.

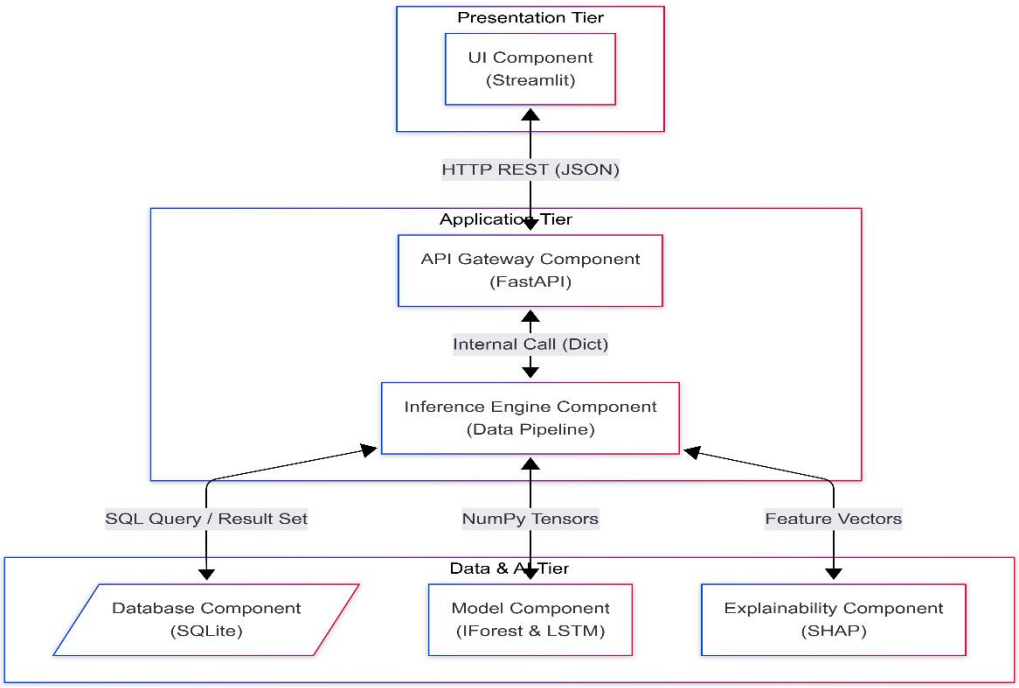

Figure 2.1: Internal Component Architecture.

2.1 Execution Pipeline

The execution pipeline follows a strict, sequential runtime flow managed by the backend microservice:

1. **Data Ingestion:** FastAPI receives the incoming claim payload from the Streamlit UI.
2. **Context Retrieval:** Historical provider state is loaded from SQLite.
3. **Feature Computation:** Raw data is transformed into static and temporal matrices.
4. **Inference:** The Hybrid Model generates dual anomaly scores.
5. **Ensemble Scoring:** Scores are combined and normalized.
6. **Thresholding:** The system applies decision logic.
7. **Explainability:** SHAP is triggered conditionally based on the threshold.

3. INPUT/OUTPUT DATA STRUCTURES & CONTRACTS

3.1 Incoming Transaction Payload (API Input)

The system receives data via the frontend interface endpoint. The JSON payload is strictly typed to match the Streamlit core transaction details and clinical history inputs:


```
{
 "transaction_id": 89509112315050,
 "provider_id": "2300YM",
 "claim_type": "Outpatient",
 "billing_amount": 947.25,
 "deductible_context": 0.0,
 "coinsurance_context": 40.0,
 "service_start": "2008-12-27 09:00:00",
 "service_end": "2008-12-27 10:00:00",
 "patient_age": 45,
 "diagnosis_context": "V222",
 "has_diabetes": 1,
 "has_chf": 0,
 "has_cancer": 0,
 "has_copd": 1,
 "primary_procedure": "0",
 "treatment_group": "0",
 "patient_id": "B55D152A96663E01",
 "admission_date": "2008-12-27",
 "gender_context": 1,
 "demographic_context": 1,
 "primary_service": "0"
}
```


3.2 Provider Context (Database Fetch)

Upon receiving the payload, the system queries the SQLite database to fetch the provider's historical behavior, specifically targeting:

- Historical billing amounts.
- Claim timestamps for velocity tracking.
- Patient demographic distributions and historical chronic condition frequencies.

3.3 API Response Payload (System Output)

The inference result returned to the frontend dashboard contains the system's final verdict, directly populating the "Evaluation Result" panel:

{

"transaction_id": 89509112315050,

"provider_id": "2300YM",

"anomaly_score": 1.7426,

"alert_zone": "Suspicious",

"is_normal": "No",

"reason": "Suspicious anomaly detected. Primary drivers indicate that claim amount is materially above the provider baseline.",

"top_drivers": [

"Claim amount is materially above the provider baseline.",

]

}

4. FEATURE CONSTRUCTION LOGIC

Raw transactional data lacks the necessary mathematical context for deep learning. The Feature Engine dynamically recomputes the historical state during inference by merging the incoming payload with the SQLite provider history.

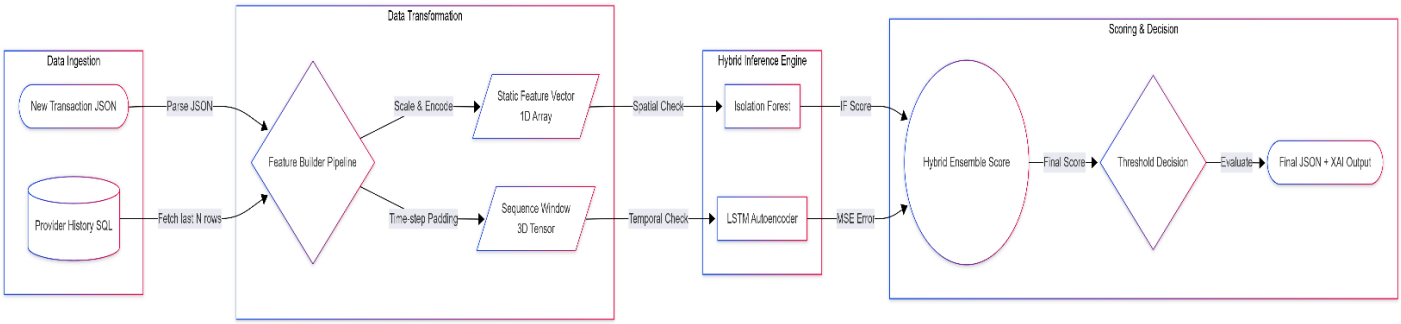

Figure 4.1: Feature Transformation Data Flow.

4.1 Static Features (Spatial Context)

- billing_amount_log: Log-transformed version of the current billing amount after clipping extreme outliers.
- deductible_context: Deductible amount associated with the submitted healthcare claim.
- coinsurance_context: Coinsurance amount associated with the submitted healthcare claim.
- provider_avg_billing: Historical average billing level for the specific provider.
- provider_std_billing: Historical billing variability for the specific provider.
- provider_deviation_score: Standardized deviation of the current claim amount from the provider's historical billing behavior.
- service_duration: Number of days between service_start and service_end.
- age: Patient age derived from patient context and service date.
- rolling_mean_7: Seven-transaction rolling mean of provider billing behavior.
- rolling_std_7: Seven-transaction rolling standard deviation of provider billing behavior.
- has_diabetes: Binary indicator showing whether the patient has a diabetes history.
- has_chf: Binary indicator showing whether the patient has a congestive heart failure history.
- has_cancer: Binary indicator showing whether the patient has a cancer history.
- has_copd: Binary indicator showing whether the patient has a chronic obstructive pulmonary disease history.
- patient_hoarding_index: Indicator of unusually concentrated patient-provider interaction behavior over time.
- dx_px_combo_spike: Measures unusual deviation between diagnosis-procedure combinations and expected provider billing behavior.

4.2 Temporal Features (Sequential Context)

- billing_amount_log: Log-transformed billing amount used in temporal sequence modeling.
- time_gap: Time difference between the current claim and the provider's previous claim, measured in hours.
- services_per_day: Total number of claims submitted by the same provider on the same service date.
- rolling_mean_7: Seven-transaction rolling mean representing the provider's short-term billing trend.
- rolling_activity_7: Seven-transaction rolling average of provider claim activity, used to capture short-term operational intensity.

These temporal features are arranged into fixed-length sequential windows of 7 transactions and are used by the LSTM autoencoder to detect unusual provider behavior over time.

5. Machine Learning Micro-Architecture

5.1 Static Model Logic (Isolation Forest)

The Isolation Forest evaluates the static feature vector to detect irregular claim behavior such as abnormal billing patterns, provider deviation, and unusual clinical context.

- **Input Vector (X_static)**: A 1D feature array containing billing_amount_log, deductible_context, coinsurance_context, provider_avg_billing, provider_std_billing, provider_deviation_score, service_duration, age, rolling_mean_7, rolling_std_7, has_diabetes, has_chf, has_cancer, has_copd, patient_hoarding_index, and dx_px_combo_spike.
- **Processing**: The vector is scaled using a pre-fitted RobustScaler.
- **Evaluation**: The system applies decision_function() and converts the result into a static anomaly signal used in hybrid scoring.

5.2 Temporal Model Logic (LSTM Autoencoder)

The LSTM Autoencoder evaluates provider behavior over time to detect temporal irregularities such as sudden claim frequency changes and unusual short-term billing patterns.

- **Input Tensor (X_seq)**: A 3D sequence window of the last 7 provider transactions using billing_amount_log, time_gap, services_per_day, rolling_mean_7, and rolling_activity_7.
- **Processing**: The sequence is scaled using a dedicated RobustScaler.
- **Evaluation**: The autoencoder reconstructs the input sequence, and the reconstruction error (MSE) is used as the temporal anomaly score.

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(X_i - \hat{X}_i)^2$$

- **Edge Case Handling (Cold Start)**: If a provider has fewer than 7 historical transactions up to the incoming claim, the temporal branch is disabled and the system falls back to the static Isolation Forest signal only.

6. EXPLAINABILITY TRIGGER (XAI)

To solve the "Black Box" problem, the system utilizes SHAP (SHapley Additive exPlanations). This module is conditionally executed to preserve computational resources and minimize API latency.

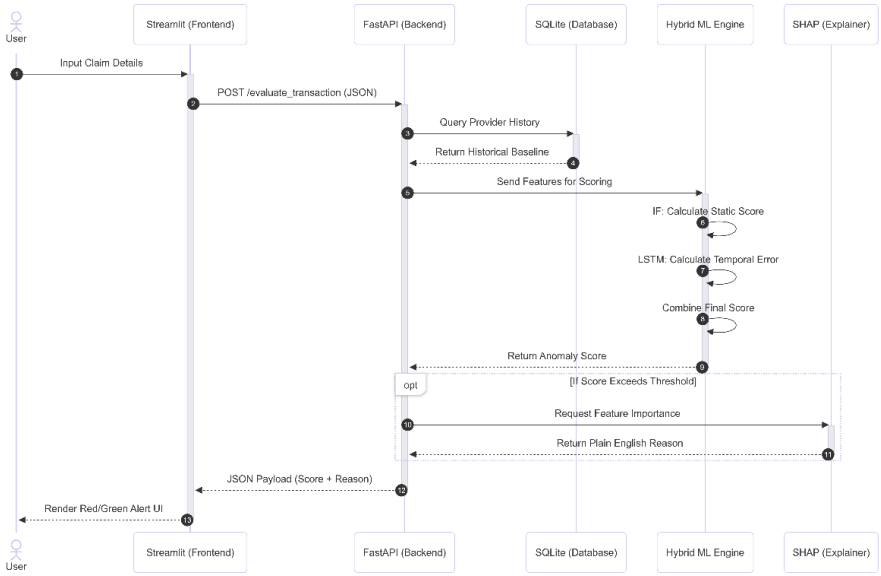

Figure 6.1: Runtime Sequence Interaction.

6.1 Conditional Execution Logic

SHAP-based explanation is generated for anomalous claims. Suspicious non-anomalous claims receive review-oriented explanation text, while normal claims return a compact normal-pattern explanation. This keeps explanation useful without adding unnecessary overhead to every claim

6.2 Output Formatting

The raw mathematical contributions are mapped to human-readable strings for the frontend UI:

- Normal Flow (is_anomaly = 0):
  Claim pattern is within the expected range. Normal behavioral pattern observed.
- Suspicious Flow:
  Pattern is near the anomaly boundary. Manual review is recommended due to mild deviations from expected behavior.
- Anomaly Flow (is_anomaly = 1):
  Fraud alert triggered. Anomalous behavior detected, driven by relative billing spike, deviation from age-group baseline average, and elevated total billing amount.

6.3 Error Handling

- The backend uses FastAPI validation, returning HTTP 422 for missing or invalid inputs and HTTP 500 for internal inference errors.
- In batch processing, failed rows are returned with alert_zone = "Processing Failed" and batch_status = "Failed" instead of being dropped.
- If provider history is insufficient, the system falls back to the static model, and the response notes that temporal analysis was inactive.

7. HYBRID SCORE COMPUTATION & DECISION LOGIC

The outputs from the static and temporal models exist on different mathematical scales. Before ensemble scoring, both metrics undergo Z-score normalization.

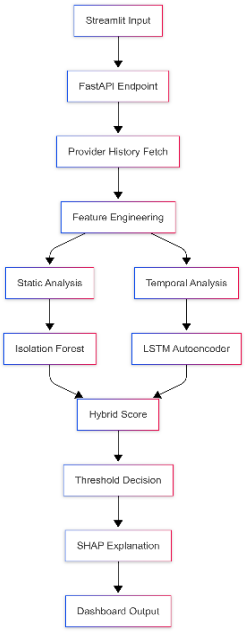

Figure 7.1: Internal Pipeline Scoring Flow.

7.1 Normalization and Fusion

The final combined anomaly score is calculated using a weighted formula:

$$Hybrid_Score = (\alpha \times Static_Score) + (\beta \times Temporal_Score)$$

7.2 Threshold Application

The system applies a hard threshold derived from the 92.5th percentile of the training distribution:

```
# Decision Logic for Frontend Rendering
if Hybrid_Score > Global_Threshold:
    is_anomaly = 1
    status_ui = "HIGH RISK ANOMALY DETECTED!"
else:
    is_anomaly = 0
    status_ui = "Normal Transaction Verified."
```

8. DEPLOYMENT & DOCKER CONFIGURATION

To ensure the system is reproducible, scalable, and environment-agnostic, the architecture is deployed using Docker containerization. The system is split into distinct microservices managed by Docker Compose.

8.1 Containerization Strategy

The project utilizes two primary Docker containers to separate the presentation layer from the heavy computational ML layer.

1. **Backend Container (FastAPI + ML Engine):**
    - **Base Image:** python:3.9-slim
    - **Responsibilities:** Hosts the SQLite database connection, loads the .pkl and .h5 model weights into memory, processes incoming JSON payloads, computes hybrid scores, and generates SHAP outputs.
    - **Exposed Port:** 8000
2. **Frontend Container (Streamlit UI):**
    - **Base Image:** python:3.9-slim
    - **Responsibilities:** Serves the interactive web dashboard, captures user inputs (like Chronic Conditions and Diagnosis Codes), and handles HTTP POST requests to the backend.
    - **Exposed Port:** 8501

8.2 Microservice Networking

A docker-compose.yml file defines the virtual network bridging the two services. The frontend container is configured to communicate with the backend container via an internal Docker network, ensuring that the ML API is not directly exposed to the public internet, thereby mimicking enterprise security standards.

Environment Variables:

- BACKEND_URL=http://backend:8000/evaluate_transaction
- MODEL_VERSION=v1.2_hybrid

9. CONCLUSION

This Low-Level Design defines a highly robust, production-ready microservice architecture for detecting healthcare transaction anomalies. By moving away from monolithic scripts and embracing a decoupled FastAPI backend and Streamlit frontend, the system mirrors professional enterprise standards.

The programmatic logic detailed in this document—specifically the dynamic construction of the spatial and sequential feature matrices, the weighted ensemble combination of the hybrid model outputs, and the application of a 92.5th percentile global threshold—ensures that the system is not only computationally efficient but also highly accurate. Furthermore, the conditional execution of SHAP explainability guarantees computational resources are preserved during normal transactions.

By following the specifications laid out in this LLD, the resulting implementation will successfully solve the dual challenges of detecting complex, temporal fraud patterns while maintaining the strict transparency (XAI) required in the healthcare sector.